



TED UNIVERSITY

CMPE 491 / SENG 491 Senior Project

**Center of Gravity Detection, Information, and Balancing System in Aircraft
High Level Design Report**

29.12.2025

Team Members:

Sude İpekci, Computer Engineering

Yasin Anber, Computer Engineering

Selin Göç, Computer Engineering

Elif Sude Memiş, Software Engineering

Supervisor:

Prof. Dr. Hakkı Gökhan İLK

Jury Members:

Dr. Ali BERKOL

Dr. Ayşe Yasemin SEYDİM

Table of Contents

1. Introduction	2
1.1 Purpose of the System.....	2
1.2 Design Goals.....	2
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 System Overview.....	4
2. Existing Solutions and Limitations	4
2.1. Analysis of Existing CG Measurement and Balancing Methods	4
2.2. Limitations of Current Solutions	5
2.3. The Necessity for a New Embedded System Design.....	5
3. Proposed Embedded System Architecture	5
3.1 Overall Embedded Architecture.....	5
3.2 Hardware Subsystem Decomposition	6
3.3 Software Architecture	9
3.3.1. Sensor Acquisition and Signal Processing	10
3.3.2. Center of Gravity (CG) Calculation Function	10
3.3.3. Servo Control and Balancing Algorithm.....	10
3.4 Hardware–Software Integration	11
3.5 Data Communication and Monitoring	12
3.6 Control Logic and System Flow.....	12
3.6.1 Signal Acquisition and Processing Stages of a Single Load Cell	12
3.6.1.1 Failure Analysis and Mitigation (Safe Behaviors).....	15
3.6.2. Main Control Loop.....	15
3.6.2.1 Error Handling and Fail-Safe Behaviors	16
3.7 Boundary Conditions and Fault Handling	17
4. Subsystem Functional Services	17
5. Bill of Materials	18
6. Glossary	19
7. References.....	19

1. Introduction

1.1 Purpose of the System

An embedded system is used to automatically determine and maintain the aircraft center of gravity (CG) by processing real-time weight data from strain gauge-based load sensors placed under the landing gear. Management of CG is a critical requirement during design and operation of aircraft, as exceeding or falling short of allowable limits directly affects flight stability, safety, controllability, and fuel efficiency [11].

From a technical standpoint, the center of gravity problem consists of accurately measuring distributed loads acting on the aircraft structure and determining the resultant position of CG with respect to some defined datum point. Changes in payload, fuel distribution, or configuration will result in shifts that must be detected quickly that's why appropriate actions can be taken before unsafe flight conditions develop [11].

In this scenario, the embedded system acts as a computational and control unit that connects physical measurements with corrective actions. It continuously acquires analog signals from strain gauge-based load cells, performs signal conditioning and digital processing, and runs algorithms to calculate current CG coordinates. The difference between calculated CG and optimal reference values defined by the system will indicate if corrective action is needed [12][13].

The expected inputs to the system are real-time analog signals representing weight measurements obtained from multiple load sensors. These signals reflect forces applied at each landing gear contact point. The system processes such inputs into outputs that include calculated CG position as well as control commands for servo motors. Upon detection of an imbalance, servo motors reposition a movable balancing mass in order to restore CG to its optimal location forming a closed-loop control mechanism [14][15].

The general goal is to mechanize the finding and fixing of CG, lessen the need for manual balancing steps, cut down on mistakes by people, and make reactions quicker and more precise in managing weight and balance [20]. By bringing together sensing, computing, and action-taking in a small, embedded platform, the system helps make planes safer, work better, and perform well.

1.2 Design Goals

The primary design objective of the Center of Gravity Detection, Information, and Balancing System in Aircraft is to attain dependable real-time measurement and control of the aircraft center of gravity. The system must continuously acquire weight data from multiple sensors, process this data without interruption, and respond promptly to any detected deviations from the optimal CG position. Real-time operation is essential to ensure timely corrective actions and maintain system stability under changing load conditions. Another key design goal is to ensure high measurement accuracy and fast system response. The system should minimize measurement errors through proper sensor selection, signal conditioning, and data processing techniques while achieving a short response time between CG deviation detection and corrective actuation. This balance between precision and responsiveness is critical for effective CG management and safe operation [19]. The system should also be low cost with ease of prototyping in mind using commercially available components that are not expensive so that rapid

development testing iteration can take place easily.

Finally, the design requires modular hardware and software architecture. The system consists of separate modules like sensor acquisition, data processing, control logic, actuation, and user interface. This modularity improves scalability as well as makes debugging/maintenance easier while allowing individual parts to be changed or substituted without affecting the total system performance; thus, supporting future additions/improvements.

1.3 Definitions, Acronyms, and Abbreviations

Definitions

- **Center of Gravity:** That point within an aircraft or other body, where the total weight of the aircraft or body may be considered to act. The location of the CG of an aircraft is critically important in determining its stability, controllability, and performance.
- **Load Cell:** Basically, it is a kind of force measurement sensor that converts mechanical force into an electrical signal. In this project, use strain gauge–based load cells to measure the weight applied at each landing gear point.
- **Strain Gauge:** A sensor element whose electrical resistance changes when it is mechanically strained. The strain gauges are very often used inside the load cells to detect the force or weight.
- **Balancing Mechanism:** A mechanical system shifting weight distribution by moving a movable mass to the proper center of gravity deviations.
- **Embedded System:** A specialized computer system designed for specific control and monitoring functions in the context of a larger mechanical or electrical system.
- **Real-time operation** is the capability of a system to process inputs and generate outputs in a time constraint guaranteed by the system, making it a predictable timely response:
- **Signal Conditioning:** The process of amplifying, filtering, and converting the raw sensor signal into a form suitable for digital processing.
- **Closed-Loop Control:** A control that is based on the principle of continual monitoring of system outputs and feeding back information to automatically make adjustments in system behavior.

Acronyms and Abbreviations

- **CG** – Center of Gravity
- **MCU** – Microcontroller Unit (Arduino)
- **HX711** – 24-bit ADC & PGA module
- **PWM** – Pulse Width Modulation
- **PGA** – Programmable Gain Amplifier
- **HLD** – High-Level Design
- **MCU** – Microcontroller Unit
- **PWM** – Pulse Width Modulation
- **UI** – User Interface
- **I/O** – Input / Output
- **PCB** – Printed Circuit Board
- **ADC** – Analog-to-Digital Converter

1.4 System Overview

The system will be an integrated embedded system that deals with real-time sensing, processing, and actuation in maintaining the optimal boundaries of the aircraft center of gravity. The system will involve a

formal data flow that starts with real-world measurements and ends with an appropriate mechanical reaction.

Stage I: The load sensors containing strain gauges installed beneath the landing gears of the aircraft measure the force at each point of contact. The sensors generate a low-level analog signal corresponding to the measured force. The analog signal is transmitted to an analog-to-digital converter for amplification, filtering, and conversion into a digital signal.

The data from the digitized sensor is received by the microcontroller unit (MCU). This MCU basically serves as the central processor of the concerned system. This microcontroller runs algorithms based on the data received from the sensor and compares the calculated data with the certain predefined optimal values. All these tasks are carried out on a continuous basis so that the monitoring can be done in real time.

When there is a deviation from the optimal center of gravity, it triggers the production of control signals by the MCU, which are then provided to the servo motors, which actuate the balancing mechanism by repositioning the weight along the desired axis or axes. The effect is that it balances the weight distribution by re-adjusting the center of gravity, thereby forming a closed-loop control system [20].

From a software point of view, it can be noted that the system architecture is divided into modular subsystems like data acquisition from sensors, processing, control logic, actuation control, as well as handling user interfaces. The hardware as well as software aspects are highly integrated and modular, to enable each of these systems to function as individual systems as well as be part of a larger system.

2. Existing Solutions and Limitations

2.1. Analysis of Existing CG Measurement and Balancing Methods

Traditional methods used for center of gravity (CG) management in the aviation industry possess significant technical limitations regarding operational flexibility and dynamic intervention. The most common practice, pre-flight static weighing, involves placing the aircraft on fixed scales or load cells integrated into the landing gear before takeoff. However, this approach only reflects a "snapshot" of the aircraft's state at the time of measurement and lacks the capability to track real-time load changes during flight. Weight shifts occurring after takeoff are generally managed through theoretical calculations performed manually by pilots or ground personnel using "Load and Trim Charts." This manual process is highly susceptible to human error, particularly in critical variables such as dynamic cargo placement or unexpected shift in payloads. [20]

Furthermore, current active balancing solutions are quite limited in scope. Fuel trimming methods, typically utilized in large-bodied commercial aircraft, suffer from slow response times and are entirely dependent on the aircraft's instantaneous fuel availability. Such systems are not applicable to smaller platforms, such as UAV models or light aircraft. In these small-scale platforms, balancing is generally achieved using fixed ballast weights. This method offers no adjustment mechanism to compensate for dynamic CG shifts during flight; moreover, these "dead weights" increase the aircraft's gross weight, which directly compromises fuel efficiency and overall flight performance. [16]

2.2. Limitations of Current Solutions

The technical framework of traditional weight and balance management is currently hindered by several

operational bottlenecks:

- **Static Scope vs. Dynamic Reality:** Most tools are designed for "pre-flight" scenarios, providing only a static snapshot of the aircraft's weight. This approach fails to account for real-time center-of-gravity (CG) shifts caused by payload movement or fuel consumption during flight.
- **The Human Factor:** Relying on manual data entry and physical calculations is inherently slow and invites significant risk. Even minor inaccuracies in these manual processes can jeopardize flight safety by causing severe aerodynamic instability.
- **Operational Drag:** The requirement for manual weighing and physical adjustments leads to an unnecessary extension of pre-flight preparation times. This creates a major hindrance in fast-paced testing cycles and rapid prototyping environments.
- **Absence of Active Control:** Existing solutions for small aircraft and UAVs act strictly as measurement tools rather than controllers. They lack the mechanical intelligence or systemic capacity to autonomously detect and rectify an imbalance once the aircraft is in motion.

2.3. The Necessity for a New Embedded System Design

The limitations of existing methods necessitate a transition from passive measurement to **active, autonomous stabilization**. A new embedded system architecture is required for several key reasons:

- **Real-Time Autonomy:** There is a critical need for a system that provides "Closed-Loop" control. By integrating 24-bit HX711 ADC modules with Arduino-based microcontrollers, this project enables the processing of microvolt-level sensor data into immediate mechanical action.
- **Precision Actuation:** A new design is required to move away from fixed ballast. The proposed "lead screw" and servo motor mechanism allow for millimetric precision in shifting a counter-balance weight to the exact (X, Y) coordinates needed for stability.
- **Digitalization and Safety:** Automating the detection and correction process removes the human error factor, ensuring that the aircraft remains at its optimum CG point regardless of internal or external load changes.
- **Cost and Portability:** Existing high-end aviation sensors are expensive and heavy. This new design focuses on a low-cost, lightweight, and modular structure that can be easily integrated into various platforms, such as cargo drones and UAVs.

3. Proposed Embedded System Architecture

3.1 Overall Embedded Architecture

The Automatic Center of Gravity Detection, Information, and Balancing System is built on an integrated embedded system architecture consisting of sensor, signal processing, embedded software, and actuator components. The system architecture aims to process data from strain gauge load cells located under each wheel of the aircraft in real time and to automatically calculate the center of gravity.

Low-level analog signals obtained from load cells are amplified and converted into digital form via the HX711 module [3][5]. These digital data are read by the Arduino Uno microcontroller at specific sampling intervals. The digital data are processed by C/C++ based embedded software algorithms [7]. The software architecture consists of sensor reading, data filtering, center of gravity calculation, balancing decision mechanism and serial communication layers.

When the system detects deviations in the center of gravity, it restores the aircraft's balance to the optimum point by moving the balancer weight along the X and Y axes with the help of servo motors and

a worm gear mechanism [9]. The balancing system mechanism was designed inspired by the movement of 3D printers along the x and y axes [8]. This architecture was designed considering the criteria of real-time control, modularity and expandability.

3.2 Hardware Subsystem Decomposition

The embedded system developed within the scope of the project consists of hardware subsystems separated according to their functions. This separation increases the modularity of the system and facilitates maintenance and testing processes.

Subsystems:

- Load Sensing Subsystem (Load Cell + Wheatstone Bridge)
- Signal Conditioning and ADC Subsystem (HX711)
- Embedded Control Unit (Arduino)
- Balancing Mechanism (Servo + lead screw)
- Serial Communication and Monitoring Subsystem (USB–UART–SPI)

The first subsystem is the load-sensing subsystem. It consists of strain gauge type load cells to be placed under each wheel of the aircraft. These sensors convert the mechanical deformation caused by the force (N) acting on them into an electrical signal [6]. The strain gauge resistor elements in the internal structure of the sensor create a resistance change with the applied force. This resistance change is detected as a voltage difference using the Wheatstone bridge circuit principle, thus enabling precise force measurement [1]. In this project, Full-Bridge type load cells will be used. The full-bridge (figure1) structure has four active strain gauges, all of which are placed on the four arms of the bridge circuit [2]. This structure has high signal sensitivity. It is also stable against noise and temperature changes. The full-bridge circuit works directly with the HX711 amplifier module. It does not require additional resistors or balance elements. Each full-bridge load cell will be placed under the relevant wheel. It enables the measurement of the load on the wheel under which it is located. A total of three full-bridge load cells will be used for the three-wheeled aircraft platform. This will allow the system to detect the force components on each wheel separately, thereby calculating the total weight and the position of the center of gravity (X, Y coordinates) with high accuracy.

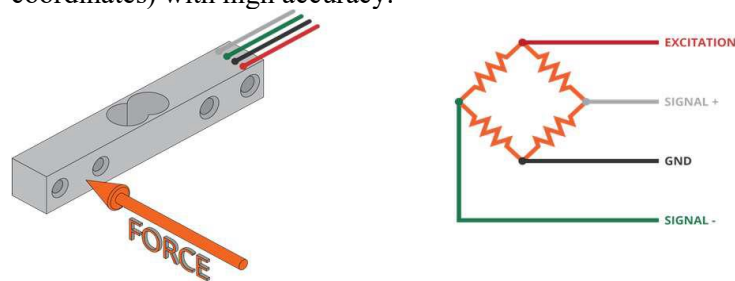


Figure 1: Full-bridge load cell

The second subsystem is the signal conditioning and analog-to-digital conversion subsystem. The HX711 (figure2) converts low-level analog signals from load cells into high-accuracy digital data. It is an analog-to-digital converter (ADC) and differential amplifier integrated circuit with 24-bit resolution [5]. Thanks to these features, it works directly with full-bridge sensors, especially strain gauge type sensors. The programmable gain amplifier (PGA) inside the module can amplify the microvolt-level signal from the load cell by up to 32, 64, or 128 times [3]. In this way, the system can accurately detect even very small load changes. This high-gain measurement capability of the HX711 minimizes noise before converting

the analog signal to digital format and increases measurement accuracy. Within the scope of the project, the HX711 module (figure4) converts the strain data from each load cell sensor into digital signals and transmits them to the Arduino microcontroller. Arduino runs an algorithm that calculates the total weight and center of gravity by reading this data at specific sampling intervals. Communication between the HX711 and Arduino is carried out via serial digital communication (2-pin interfaces: DT and SCK) (figure3). This provides ease of connection and low energy consumption. The reason for choosing the HX711 module is its high resolution, low noise, and low cost. It is a measurement system that is directly compatible with strain gauge sensors. The module's ability to operate with 5V or 3.3V ensures seamless integration into our Arduino-based embedded system design [4].

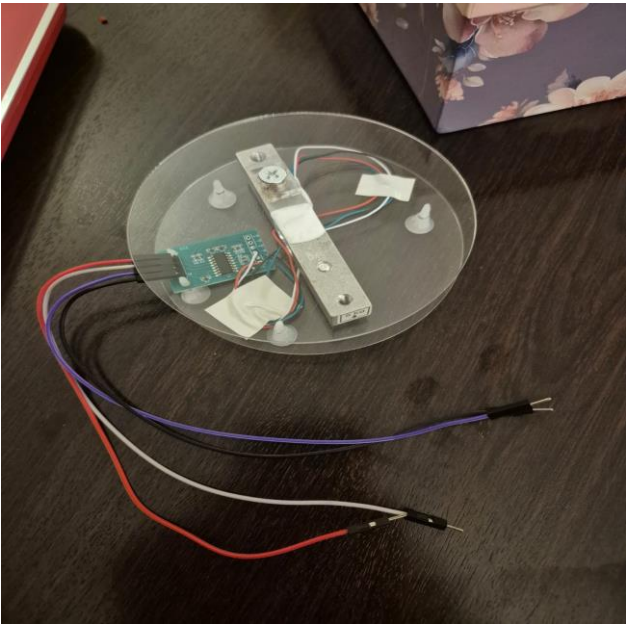
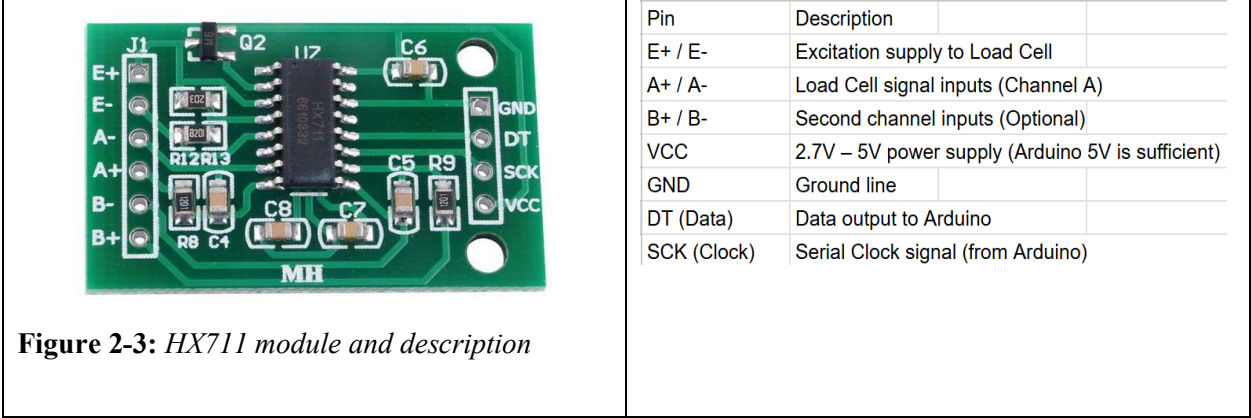


Figure 4: *The load cell was connected to the HX711 and installed inside the platform.*

(This image shows our connection between the load sensor and the HX711 module.)

The third subsystem is the Arduino Uno-based embedded control unit. The Arduino Uno is built on the ATmega328P microcontroller and offers an ideal platform for embedded systems [7]. It has an 8-bit RISC core running at 16 MHz. It can process 24-bit weight measurement data from the HX711 in real time. The Arduino Uno allows for real-time monitoring of weight distribution in the balancing system.

The Arduino Uno has 14 digital I/O pins, 6 analog inputs, and supports serial communication protocols such as UART, SPI, and I2C [10]. This allows for easy integration with sensors and other modules. It has low power consumption. The Arduino's compact design makes our project portable and energy efficient. Its programmability with the Arduino IDE allows for software development in embedded systems. Thanks to its extensive community support and readily available libraries, it will provide rapid integration with the HX711 and other sensors. The prototype development and optimization process is accelerated. The Arduino Uno (figure5) will be the brain of our aircraft balancing system design.

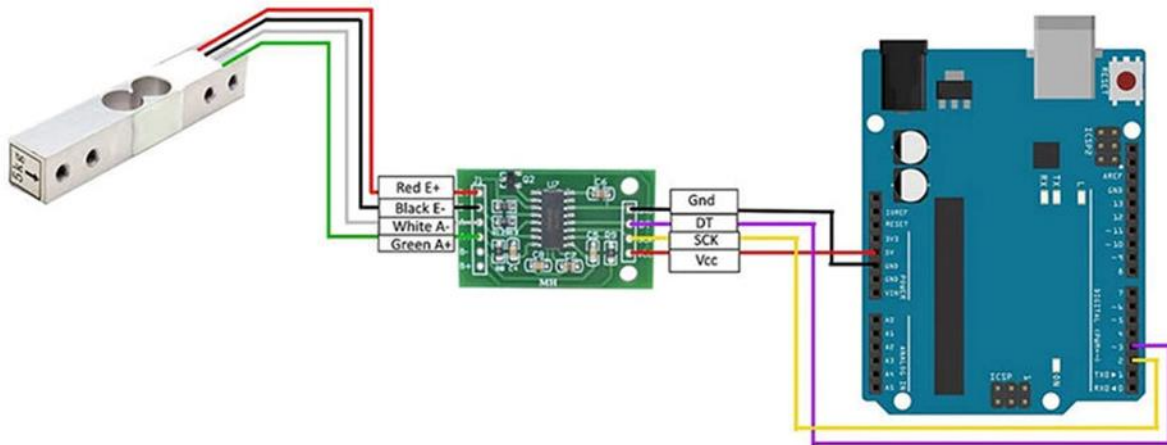


Figure 5: Load Cell, HX711, and Arduino Uno: Connection Schematic and Pin Map

The fourth subsystem is the balancing mechanism. The most important design in the project is to maintain the center of gravity of the aircraft at the optimum point. To balance the center of gravity, a system design will be created that allows the counterweight to move precisely. The system includes a counterweight, two 360° rotatable servo motors, and two lead screw mechanisms (figure6). Servo motors are precision control elements capable of rotating at specific angles, and each rotates the lead screw to which it is attached, converting rotational motion into linear motion [9]. In this way, the counterweight is shifted in a controlled manner along the X and Y axes, and the center of gravity can be adjusted with millimeter precision. The screw pitch determines how far the weight will advance when the screw is rotated one full turn. The system prevents the weight from returning under external forces, keeping its position fixed. The servo motor on the X-axis moves the weight horizontally, and the servo motor on the Y-axis moves it vertically. The system detects deviations in the center of gravity and aims to maintain the center of gravity of the aircraft at the optimum point by moving the counterweight to the calculated (X, Y) coordinate. In this way, the aircraft aims to reach a stable equilibrium position again. We designed this design for our project by being inspired by the way 3D printers move in X and Y coordinates [8].

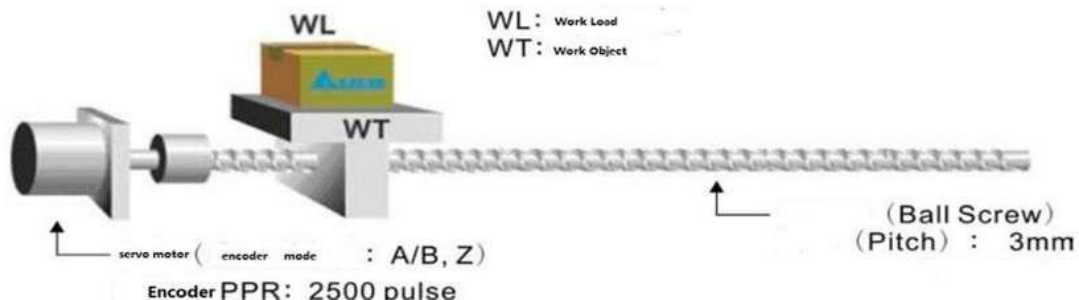


Figure 6: *Sample Design of Balancing System: X-Axis Linear Actuator Configuration Motor and Lead Screw*

The fifth subsystem, the serial communication and monitoring subsystem, ensures the reliable and organized transfer of measurement data obtained by the embedded system to the computer environment. Very low-level analog signals from the load cells are amplified by the HX711 module and converted into 24-bit digital data. Data transfer between the HX711 and the Arduino Uno is achieved via a two-wire synchronous communication structure using the DT (Data) and SCK (Clock) lines. This communication is similar to the SPI (Serial Peripheral Interface) protocol. The Arduino reads the raw weight data from the HX711 at specific time intervals via these lines and processes it within the software.

The weight, load distribution, and center of gravity (X, Y) coordinates calculated on the Arduino Uno are then transmitted to the computer via UART-based serial communication. This allows for real-time monitoring of the system's status. Through serial port monitoring software or a custom interface developed on the computer side, the measurement results are presented to the user both digitally and visually. This subsystem is not only for monitoring purposes; It also forms a critical feedback mechanism for calibration processes, evaluation of test scenarios, and verification of system performance.

3.3 Software Architecture

The software of the system is designed as a real-time, closed-loop control platform developed in C/C++ within the Arduino environment. The architecture follows a sequential logic: capturing high-resolution load data, processing it through digital filters, calculating the center of gravity (CG), and executing precise mechanical movements.

3.3.1. Sensor Acquisition and Signal Processing

The first stage involves the stable acquisition of weight data from the triple load cell configuration.

- **Data Retrieval:** Digital weight data is captured from the HX711 24-bit ADC modules via a two-wire synchronous serial interface (DT/SCK).
- **Calibration and Taring:** An initial tare() function is executed to zero the system by accounting for the aircraft's empty weight, followed by the application of calibration factors to convert raw digital signals into force units.

```
scale.set_scale(calibration_factor);  
scale.tare();
```

- **Low-Pass Filtering:** To eliminate high-frequency noise and mechanical vibrations that could lead to erratic measurements, a specialized **Low-Pass Filter library** is integrated into the code. This software-based filter ensures that only the stable, low-frequency weight data is passed to the calculation functions, providing a smoothed input for the CG algorithms.

```
Filter::LPF<float> lpf(0.001f);
```

3.3.2. Center of Gravity (CG) Calculation Function

The core mathematical module utilizes the Moment Principle to determine the aircraft's current balance point relative to the defined Datum.

- **Moment Aggregation:** The function `calculate_CG()` computes the total longitudinal and lateral moments by multiplying the filtered force from each load cell by its fixed coordinate distance from the origin.
- **Coordinate Mapping:** The final (CG_x , CG_y) coordinates are derived by dividing the total moment by the total weight (W_{TOTAL}) measured by the three sensors.

3.3.3. Servo Control and Balancing Algorithm

Once a deviation from the optimum CG is identified, the software commands the mechanical correction unit.

- **Error Analysis:** The system continuously compares the current CG coordinates against the target "optimum" coordinates to calculate the required displacement for the counterweight.
- **Mechanical Actuation:** The control function translates the required displacement into specific rotational commands for the 360-degree servo motors.
- **Lead Screw Execution:** By controlling the servos, the software drives the X and Y lead screw mechanisms, moving the balancing mass to the exact location needed to restore aerodynamic stability. The high pitch-precision of the lead screws allows the software to maintain the counterweight's position even under external pressures without back-driving.

3.4 Hardware–Software Integration

Hardware-software integration is critical for the reliable, stable, and real-time operation of the Automatic Center of Gravity (CG) Detection and Balancing System. In this context, all hardware components used in the system have been selected and integrated to be compatible with the embedded software architecture. The main objective of the integration process is to accurately, with low latency and noise-resistant transmission of physical data from the sensors to the software, processing it, and controlling the mechanical balancing elements based on this data.

The system's sensing layer uses strain gauge load cells positioned under each wheel of the aircraft to physically detect load distribution. The microvolt-level analog signals obtained from the load cells are converted into digital data by the HX711 module. This data is sent from the HX711 module to the Arduino Uno via the DT-SCK. The embedded software directly schedules this communication line, synchronously reading the raw data from the HX711 and performing scaling (calibration) and filtering operations on the software side.

The C/C++ based embedded software running on the Arduino Uno is designed with a hardware-independent modular structure. Sensor reading, data filtering, center of gravity calculation, balancing algorithm, and communication tasks are configured as separate software modules. This approach ensures that potential changes to the hardware components have a minimum impact on the software. Furthermore, the software is designed with a loop-based task structure to support real-time operation.

The calculation software determines the force values for each wheel using digital weight data from the HX711 and performs center of gravity (X, Y) calculations based on the moment principle. The results are

passed to the balancing decision mechanism by the software. At this stage, the software evaluates the calculated center of gravity deviation and generates control signals to be sent to the servo motors.

The servo motors located in the actuator layer are controlled by PWM signals generated by the Arduino. Thanks to this integration between software and hardware, the servo motors move the worm gear mechanism, precisely positioning the balancer weight along the X and Y axes. Thus, the balancing decision calculated in the software is directly applied to the physical system.

Serial communication and monitoring infrastructure were used to verify hardware-software integration. Arduino transmits the system's real-time status and calculated data to the computer via UART, providing a user interface and feedback environment for both developers and users.

```
Message (Enter to send messa
Agirlik: 9.84
Agirlik: 9.84
Agirlik: 9.84
Agirlik: 9.84
Agirlik: 9.85
Agirlik: 9.85
Agirlik: 9.85
Agirlik: 9.84
Agirlik: 9.84
Agirlik: 9.83
Agirlik: 9.84
Agirlik: 9.85
Agirlik: 9.84
Agirlik: 9.85
Agirlik: 9.84
Agirlik: 9.84
Agirlik: 9.84
```

(Measurement data of a 9.84-gram object from a serial monitor)

3.5 Data Communication and Monitoring

A serial communication infrastructure is used to monitor the measured weight and center of gravity data in the system. In our project, the Arduino Uno microcontroller will perform the measurement and processing of weight data. The HX711 module will transfer the collected digital weight data to Arduino. The connection between the HX711 and the Arduino is based on a two-wire synchronous serial communication method. For this connection, the DT (Data) pin of the HX711 module is connected to the data input pin of the Arduino, and the SCK (Clock) pin is connected to the clock pin of the Arduino [4]. This communication method performs synchronous data transfer in a way similar to the SPI (Serial Peripheral Interface) protocol. It does not include all the lines of SPI (MISO, MOSI, CS). The HX711 uses its own special two-line communication protocol. With this protocol, the Arduino receives one bit of data from the HX711 with each clock pulse, making a precise measurement with 24-bit resolution. When Arduino receives data from the HX711, C/C++ based embedded software processes this data and calculates the center of gravity. The calculations will be transmitted to the computer via UART (Universal Asynchronous Receiver-Transmitter) using the Arduino's TX (transmit)/RX (receive) pins. On the computer side, special interface software will visualize this data. Thus, the instantaneous weight distribution and center of gravity location of the system will be shown. The data is sent in a regular and processable format (e.g., CSV) to present the instantaneous weight distribution, total weight, and center of gravity coordinates of the system to the user. This monitoring structure creates an important feedback mechanism for evaluating system performance, calibration processes, and testing procedures.

3.6 Control Logic and System Flow

3.6.1 Signal Acquisition and Processing Stages of a Single Load Cell

To grasp how the system actually "senses" the aircraft's weight, we need to look at the signal's evolution through five key stages. This is where a simple physical force is transformed into the precise digital

intelligence the Arduino needs to make decisions.

1. **Physical Interaction and Mechanical Transduction:** Everything kicks off the moment the aircraft's weight settles on the sensors. This load forces the aluminum core of the load cell to undergo a microscopic, elastic "flex". As the metal subtly deforms, the internal strain gauges—firmly bonded to the structure—stretch or compress right along with it.
2. **Turning Flex into Voltage:** These physical shifts are immediately translated into electrical data. As the gauges change shape, their electrical resistance fluctuates, which unbalances the internal Wheatstone Bridge circuit. This imbalance produces a tiny differential analog voltage, measured in millivolts (mV/V), effectively acting as the electrical "signature" of the weight.
3. **Boosting the Signal (The Pre-Amp Stage):** At this point, the signal is incredibly faint—little more than a whisper that could easily be lost in a sea of electrical noise. To solve this, the HX711 module steps in with its internal Programmable Gain Amplifier (PGA). By boosting the signal 128 times, it ensures the data is loud and clear enough for the next phase.
4. **24-bit Precision Digitization:** Now that the signal is amplified, it's time to go digital. A high-resolution 24-bit Analog-to-Digital Converter (ADC) samples the analog wave, breaking it down into millions of tiny digital steps. It is this immense level of detail that grants our system its sub-gram sensitivity, which is vital for detecting even the smallest balance shifts.
5. **Clean Delivery and Digital Smoothing:** In the final stretch, the digitized information travels to the Arduino over a serial interface. But before the controller acts on it, we apply a software-based Low-Pass Filter. This acts as a "noise-canceling" layer, stripping away the jitter caused by motor hum or structural vibrations, leaving us with a smooth and dependable weight reading.

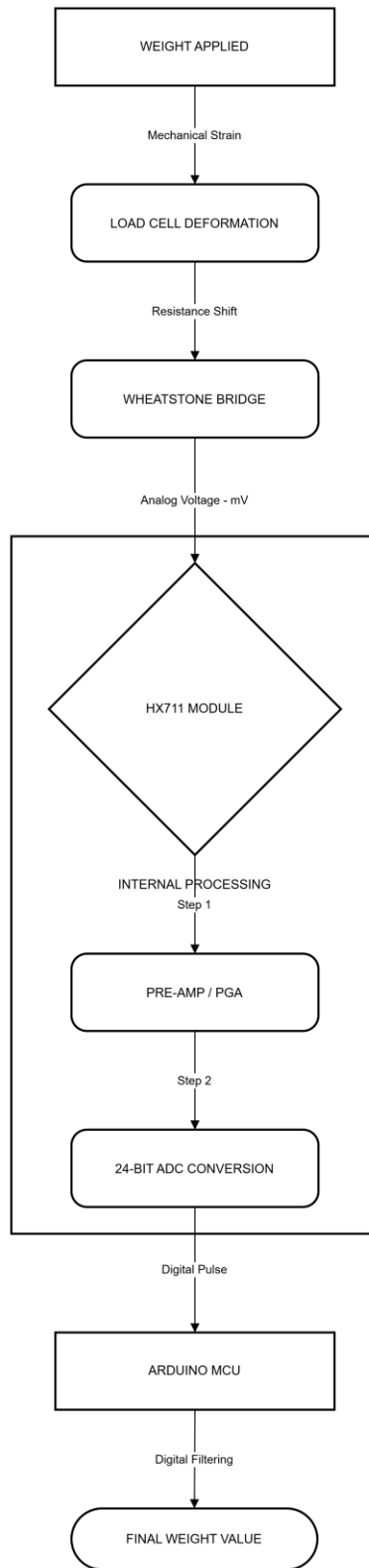


Figure 7: *Signal Conversion and Acquisition Flow of a Load Cell- HX711 Interface*

3.6.1.1 Failure Analysis and Mitigation (Safe Behaviors)

To ensure operational reliability, potential failure points and their corresponding mitigation strategies are detailed below:

Table 1: *Failure Analysis*

Failure Point	Potential Cause	Impact	Mitigation / Safe Behavior
Wiring Failure	Loose connections or broken cables.	HX711 produces "Floating" data or stops responding.	Implement <code>is_ready()</code> checks in the software to verify data flow.
Electrical Noise	EMI from nearby motors or ESCs.	Sudden, large spikes (jitter) in weight readings.	Utilize a digital Low-Pass Filter and shielded cabling.
Calibration Drift	Temperature fluctuations or mechanical fatigue.	Deviation of measured weight from the actual value.	Execute an automatic tare() protocol before every flight.
Mechanical Overload	Load exceeding the cell's capacity.	Permanent deformation and sensor damage.	Program "Overload" alerts and install physical mechanical stops.

3.6.2. Main Control Loop

To keep the aircraft in perfect equilibrium, the software runs a non-stop, closed-loop cycle. This process is essentially the "brain" of the operation, moving through four specific phases to ensure every adjustment is precise:

- **The Measurement Phase:** It all starts with the system polling all three load cells (L1, L2, L3) at the same time. This allows us to capture a real-time snapshot of the weight distribution across the landing gear. The data arrives as raw, 24-bit digital counts straight from the HX711 modules, providing the high-resolution foundation needed for the next steps.
- **The Calculation Phase:** Here, the `calculate_CG()` function takes over to find the (CG_x, CG_y) coordinates. It relies on the Principle of Moments, calculating everything relative to a fixed reference point we call the Datum. However, raw data can be jittery due to motor hum or structural vibrations. To fix this, we pass the data through a Digital Low-Pass Filter, smoothing out the "noise" before any coordinates are finalized.
- **The Decision Phase:** Once we have a clean CG position, the logic compares it against our pre-set optimum limits. If the deviation drifts past the established safety threshold, the system recognizes the imbalance. At this point, it triggers a specific correction command to bring the aircraft back in line.
- **The Action Phase:** This is where the correction actually happens. The software figures out exactly how many lead screw rotations are needed and sends Pulse-Width Modulation (PWM) signals to the 360-degree servo motors. These motors drive the counterweight toward the neutralizing position. The cycle finally wraps up once the sensors confirm the weight has reached its target and the CG is safely back in the "green zone".

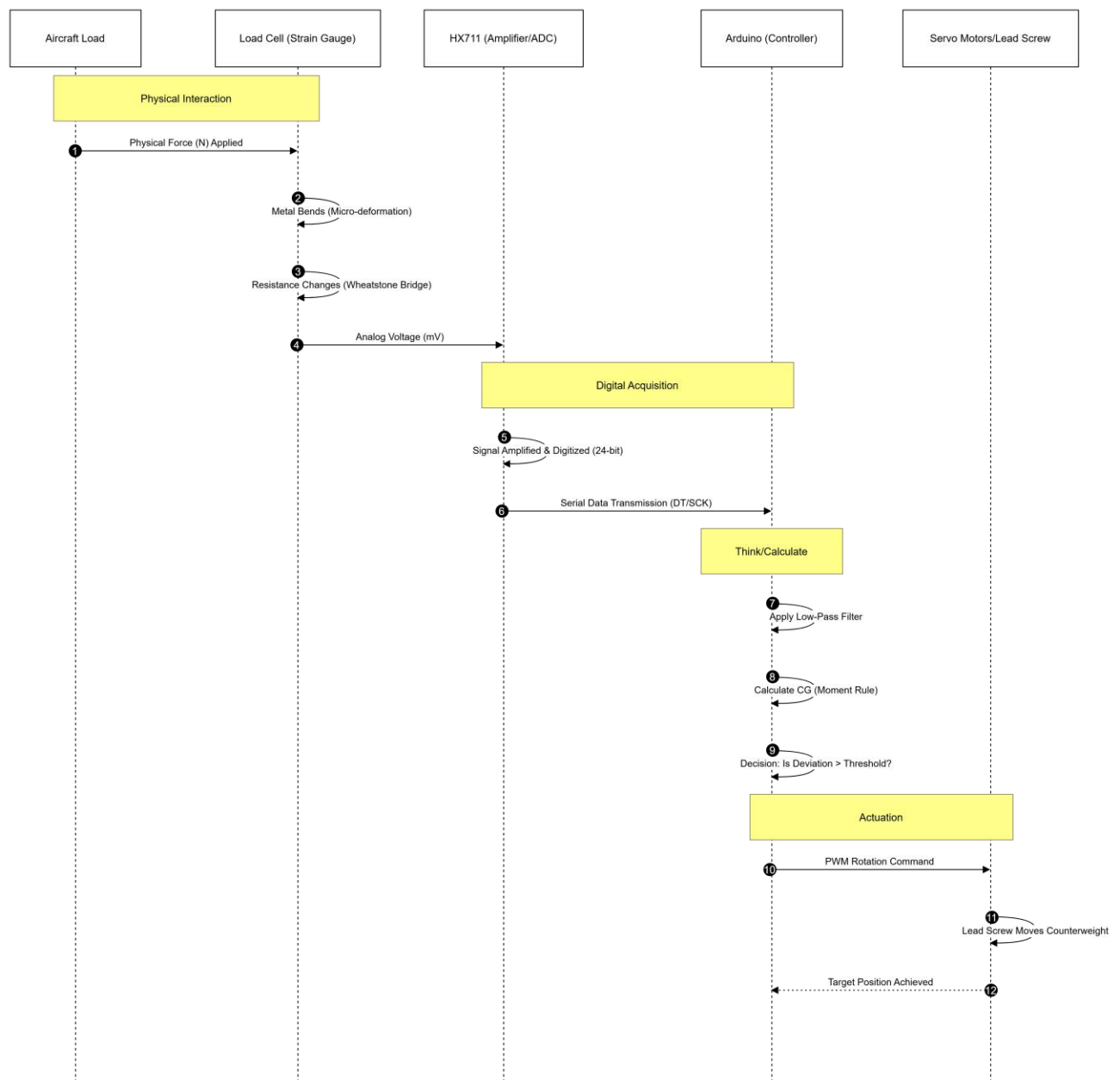


Figure 8: *Real-Time CG Control: Acquisition, Processing, and Feedback Balancing*

3.6.2.1 Error Handling and Fail-Safe Behaviors

Safety is the absolute priority in any aviation-related application. To handle the unpredictable nature of hardware in the field, our system utilizes a set of "Safe Behavior" protocols. These are essentially built-in reflexes designed to catch potential failures before they can risk the aircraft or the internal mechanism.

- **Sensor Out-of-Range Protection:** Sometimes a load cell might send back a value that simply doesn't make sense—perhaps exceeding its physical limits due to a sudden impact or a sensor malfunction. If this happens, the system doesn't try to guess the weight. Instead, it immediately locks the counterweight in its current spot to avoid any erratic or dangerous movements that could further destabilize the plane.
- **Handling Communication Timeouts:** Reliability depends on constant data flow. If an HX711 module drops out or stops responding within the expected window, the software doesn't fly blind. It triggers an instant warning and switches into "Passive Mode," holding onto the last known stable state to keep everything safe until the connection is restored.
- **Mechanical Obstruction Safeguards:** We also monitor the physical movement of the balance weights. If the servo motors start drawing too much current or fail to hit their target position, it's a clear sign of a mechanical jam or obstruction. To prevent stripping the gears or burning out the motors, the system kills power to the lead screw mechanism and alerts the operator through the control interface immediately.

3.7 Boundary Conditions and Fault Handling

The system is intended to function within certain conditions for safe and error-free operation and is capable of dealing with possible fault conditions that may arise during its execution. Such conditions are taken into account to ensure robustness and prevent any damage to hardware.

The other important boundary condition is sensor disconnection or overload detection. The system detects situations where a load sensor is disconnected, unresponsive, or generating readings that are outside its working range as an error situation. Another error situation is detected when an overload of the sensor's capacity is sensed. When this happens, the system ignores any errant sensor readings and avoids an incorrect calculation of CG and shifts to a safe operation state, where any form of balancing is disabled, and an indication is given to the user through the system interface.

Secondly, the system must address the issue of managing data that could be unstable due to noise emitted by the sensors. This could result due to the production of electrical noise, vibrations, or fluctuations in the measurements assumed by the sensors. In this case, the system will be capable of observing the level of data stability. If unstable data is detected, the system will not take any corrective measures but instead postpone CG correction until the measurements become more accurate.

It also ensures compliance with movement constraints for servo motors and mechanical aspects. Physical constraints include maximum travel, angular constraints, as well as mechanical end stop values, which are considered to avoid harming the balancing system. If the system detects a commanded position that surpasses these constraints, it ensures that movement stops at safe positions that cannot cause both stress to motor servo components as well as mechanical components.

In general, by considering sensor failures, data instability, and mechanical limitations, the system is able to provide safe functioning in abnormal circumstances.

4. Subsystem Functional Services

The software of the system is divided into a set of functional services, where each service is responsible for a specific task related to the corresponding subsystem. This service-oriented structure improves modularity, simplifies system maintenance, and enables independent development and testing of each subsystem.

Sensor Reading Service:

The Sensor Reading Service, for example, is in charge of collecting an immediate reading of the aircraft's weight using strain gauge-load sensors embedded under each of the aircraft's landing gear components. The load cells provide analog outputs, which must be converted to digital information by the HX711 module, which has a resolution sufficient to handle up to 24-bit data. The HX711 module communicates with and runs in tandem with the Arduino Uno microcontroller board via two-wire synchronization in its data and clock terminals, abbreviated as DT and SCK, respectively, as described in [4]. It guarantees that the data obtained from the sensor is legitimate and suitable for processing other services.

Center of Gravity Calculation Service:

The Center of Gravity Calculation Service is responsible for processing the digital weight information submitted by the Sensor Reading Service. By employing fixed mathematical models and knowing the positional information of the load sensors on the aircraft, the service computes the present coordinates of the center of gravity of the aircraft based on the weight information submitted by the sensors. The calculated CG points indicate the system's state and are submitted to the Balancing Control Service and the data transmission system.

Balancing Control Service:

The Balancing Control Service compares the calculated values for the center of gravity with optimal values for the center of gravity that have been set beforehand. If a discrepancy is found in the desired center of gravity, the service calculates the necessary measures to correct the system and issues corresponding balancing control commands to the actuation system. The main purpose of the service is to trigger the balancing system, if necessary, to maintain the system in the optimal center of gravity position.

Serial Data Transmission Service:

The Serial Data Transmission Service is responsible for the communication that takes place between the embedded system and the external platforms used in the monitoring process. The data concerning the weight as well as the center of gravity, after being processed by the Arduino, is then transmitted to the computer using UART through the Arduino TX and RX pins.

This system transmits data in terms of sensor values, computed CG values, and system status in an organized and interpretable form such as CSV. In the computer setup, there is specialized interface software that displays all this data, enabling real-time monitoring of both weight distribution and center of gravity positions. This data transmission system is an important aspect of system monitoring and calibration.

5. Bill of Materials

Table [2] presents a comprehensive Bill of Materials (BOM) that outlines all components required for the system's implementation in a hierarchical structure. The list provides a detailed breakdown of quantities, including the HX711 ADC modules and load cells used for data acquisition, the Arduino Uno microcontroller serving as the central processing unit, and the mechanical components such as servo motors and lead screws used for the balancing mechanism. Furthermore, specific test weights are included to facilitate system calibration and ensure measurement accuracy. This organized framework supports the project's modular design goals, streamlining the tracking of assembly, procurement, and maintenance processes.

Table 2: Bill of Materials (BOM) for the Center of Gravity Detection and Balancing System

LEVEL	DESCRIPTION	QUANTITY	STATUS
0	PRODUCT	1	
1	LAPTOP	1	
1	BASE PLATFORM	1	
2	LOAD SENSOR (Strain Gauge Load Cell – Full Bridge)	3	
3	HX711 ADC + Amplifier Module	3	
2	BASE / GROUND	1	
2	SENSOR and LAPTOP CONNECTION CABLES	1	
3	USB CABLE (Arduino – PC)	1	
3	JUMPER WIRES	1	
1	HELICOPTER / PLATFORM	1	
2	MECHANISM	1	
3	SERVO MOTOR (360°)	2	
3	LEAD SCREW	2	
3	NUT / CARRIER SLIDE	2	
3	SERVO MOUNTING BRACKET	2	
2	COUNTERWEIGHT	1	
1	EMBEDDED SYSTEM	1	
2	ARDUINO UNO	1	
2	POWER DISTRIBUTION MODULE	1	
2	5V EXTERNAL POWER SUPPLY	1	
1	TEST WEIGHTS	1	
2	9.85 GR	2	
2	352 GR	2	
2	850 GR	2	

6. Glossary

- Embedded System:**
 A dedicated computing system designed to perform specific monitoring and control tasks within a larger system.
- Real-Time Operation:**
 The ability of a system to process data and respond within predictable time limits.
- Control System:**
 A system that regulates the behavior of another system by processing inputs and generating control actions.
- Closed-Loop Control:**
 A control method where system outputs are continuously monitored and used as feedback to adjust system behavior.
- Actuator:**
 A device that converts control signals into physical motion, such as a servo motor.
- Signal Conditioning:**
 The process of amplifying, filtering, and preparing sensor signals for digital processing.
- Feedback:**
 Information returned from the system output to the controller to ensure stable and accurate operation.

7. References

- [1] Delta Motion. (n.d.). Load cell fundamentals. Delta Computer Systems, Inc. Retrieved November 6, 2025, from https://deltamotion.com/support/webhelp/rmctools/Content/Controller_Features/Transducer_Basics/Load_Cell_Fundamentals.htm
- [2] Diah, M. H. M. (2025). Advancing strain gauge technology: Design, integration, and industrial applications. Journal of the TSSA, (230), 144. Retrieved from <https://tssa.my/index.php/jtssa/article/view/230/144>
- [3] AVIA Semiconductor. (n.d.). HX711: 24-Bit Analog-to-Digital Converter (ADC) for Weigh Scales[Datasheet]. SparkFun Electronics. https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf
- [4] Santos, R., & Santos, S. (t.y.). Arduino with load cell and HX711 amplifier (digital scale). Random Nerd Tutorials. <https://randomnerdtutorials.com/arduino-load-cell-hx711/>
- [5] Bitfoic. (2023, 15 Eylül). Explore HX711–24 Bit ADC: Pinout, Pin configuration, Datasheet. https://www.bitfoic.com/components/explore-hx711%E2%80%9324-bit-adc-pinout-pin-configurationdatasheet?id=117&utm_source
- [6] Latha, N. A., & Murthy, B. R. (2017). Arduino based weighting scale using load cell. International Journal of Scholarly Research in Science and Technology, 3(6), 704. <https://ijsrst.com/paper/2443.pdf>
- [7] Arduino Documentation. (n.d.). UNO R3 – Arduino UNO is a microcontroller board based on the ATmega328P. Retrieved November 9, 2025, from <https://docs.arduino.cc/hardware/uno-rev3>
- [8] Gregurić, L. (2023, 11 Ağustos). 3D printer axis: The basics simply explained. All3DP. <https://all3dp.com/2/3d-printer-axis-the-basics-simply-explained/>
- [9] Soneji, K. P., Patel, H. K., & Karkari, S. (2013). Vacuum linear feed through drive using stepper motor and lead screw. 2013 Nirma University International Conference on Engineering (NUiCONE). IEEE. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6780185>
- [10] Celal, A. (2025, 12 Eylül). Arduino Uno pinleri. Diyot.net. <https://diyot.net/arduino-uno-pinleri/>

- [12] R. C. Nelson, *Flight Stability and Automatic Control*, 2nd ed., McGraw-Hill, 1998.
https://home.engineering.iastate.edu/~shermanp/AERE355/lectures/Flight_Stability_and_Automatic_Control_N.pdf
- [13] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*, 5th ed., Springer, 2016. <https://rizkia.staff.telkomuniversity.ac.id/files/2017/11/Handbook-of-modern-sensors-physics-designs-and-applications-Fraden-Jacob-Springer-2016.pdf>
- [14] R. Pallás-Areny and J. G. Webster, *Sensors and Signal Conditioning*, 2nd ed., Wiley, 2001.
https://www.google.com.tr/books/edition/Sensors_and_Signal_Conditioning/BDmQDQAAQBAJ?hl=en&gbpv=0
- [15] Texas Instruments, *HX711 24-Bit Analog-to-Digital Converter (ADC) for Weigh Scale Applications – Datasheet*, 2019. https://www.mouser.com/datasheet/2/813/hx711_english-1022875.pdf?srltid=AfmBOoqdxMebeIxdqeyIHDCZcGv9MfEVkEIIHUE7_5cHmUMLvRE4dRGI
- [16] J. Roskam, *Airplane Flight Dynamics and Automatic Flight Controls*, DARcorporation, 1995.
https://books.google.com.tr/books/about/Airplane_Flight_Dynamics_and_Automatic_F.html?id=pWUAzy87zLYC&redir_esc=y
- [17] Murray, J. (2018). A review of human factors causations in commercial air transport accidents and incidents: From 2000–2016. [Progress in Aerospace Sciences](#)
- [18] D. A Senour, “Aircraft weight and center of gravity determination system which includes alarm, self-checking, and fault override circuitry” European Patent US3584503, June 1971.
<https://patents.google.com/patent/US3584503A/en>
- [19] Yadav, N., Singh, A. R., Gupta, A., Gandham, S., Pang, J. N. K., Chang, S.-C., & Vu, H. T., *Center of Gravity Detection, Information, and Balancing System*, U.S. Patent US12278746B2, Apr. 15, 2025.
[https://patents.google.com/patent/US12278746B2/en?q=\(sens%C3%B6r\)&oq=sens%C3%B6r](https://patents.google.com/patent/US12278746B2/en?q=(sens%C3%B6r)&oq=sens%C3%B6r)
- [20] Oren, S., Balter, J., & Ben Ari, M., “Aerospace vehicle weight and balance estimation system and method,” European Patent EP3440439A1, Feb. 13, 2019.
<https://patents.google.com/patent/US6923375?q=6339780>